

IncidentIQ RAG Integration Plan

Prepared May 10, 2026 from the IncidentIQ repository review.

Executive Summary

The current IncidentIQ v7 notebook is the working application. It owns ingestion, parsing, signal extraction, clustering, agent-style reasoning, evaluation, reporting, and the Gradio upload UI. The `src/rag` folder contains a partial FAISS and LangChain based retriever intended to plug into the notebook through the existing `retrieve_runbook_context(...)` function.

The right implementation path is to first make `src/rag` reliably feed structured runbook context into the notebook, then extract notebook logic into production modules under `src/` without changing behavior.

Current System Shape

Notebook:

- Main file: `v7-final/notebooks/Multi_Agent_DevOps_Incident_Analysis_Suite_v7.ipynb`
- Defines Pydantic contracts for log events, signals, evidence clusters, incidents, actions, and responses.
- Safely accepts `.zip`, `.jsonl`, `.log`, `.txt`, and `.json` inputs.
- Skips `ground_truth_eval_only` during runtime analysis.
- Parses raw logs into normalized events.
- Extracts deterministic incident signals with regex rules.
- Clusters evidence into incident candidates.
- Calls `retrieve_runbook_context(...)`.
- Runs classifier, timeline, RCA, remediation, critic, and action-builder agents.
- Exports Markdown, JSON, CSV, cluster CSV, and output ZIP.
- Provides a Gradio upload UI.

RAG folder:

- Main module: `src/rag/rag_retriever.py`
- Knowledge base: `src/rag/knowledge_base`
- Documents include SOPs, runbooks, known error mappings, incident reports, and production architecture notes.
- `ingest()` builds a FAISS index from text and PDF documents.
- `retrieve_rag_context(...)` accepts notebook evidence clusters as dictionaries and returns per-cluster runbook chunks.

Current Integration Point

The notebook already has the correct high-level connection:

```
run_analysis
-> parse_raw_files
-> extract_signals
-> build_evidence_clusters
-> retrieve_runbook_context
-> src.rag.rag_retriever.retrieve_rag_context
-> run_incident_agents
-> remediation_agent
-> export_results
```

If the RAG module returns a non-empty result, the notebook uses FAISS context. If RAG fails or returns {}, the notebook falls back to embedded runbooks.

Critical Gaps

1. The RAG knowledge-base path is wrong. The code points to repo-root `knowledge_base`, but the actual knowledge base is `src/rag/knowledge_base`.
2. FAISS retrieval is incorrectly gated by `OPENROUTER_API_KEY`. Local vector retrieval should work without an LLM key. Only optional synthesized answers need the key.
3. The current runtime environment does not have all RAG dependencies installed.
4. RAG chunks do not fully match the remediation agent contract. The agent expects `diagnostics`, `remediation`, `validation`, and `safety_notes`, but FAISS chunks currently return those fields as empty lists.
5. Most production folders under `src/` are still placeholders. The notebook remains the real implementation.

Detailed Connection Steps

Step 1: Fix RAG Configuration

Update `src/rag/rag_retriever.py` so the default knowledge base path is:

```
Path(__file__).resolve().parent / "knowledge_base"
```

Recommended configurable version:

```
KNOWLEDGE_BASE = Path(os.getenv(
    "INCIDENTIQ_KB_DIR",
    str(Path(__file__).resolve().parent / "knowledge_base")
))
```

Keep FAISS output under `src/rag/faiss_index` or make it configurable with `INCIDENTIQ_FAISS_INDEX_PATH`.

Step 2: Install Dependencies

From the repo root:

```
python3 -m pip install -r requirements.txt
```

Required packages include LangChain, FAISS, HuggingFace embeddings, sentence transformers, and PDF support.

Step 3: Build the FAISS Index

From the repo root:

```
python3 -c "from src.rag import ingest; ingest()"
```

Expected result: documents from SOPs, runbooks, error mappings, incident reports, and prod architecture are loaded, chunked, embedded, and saved to `src/rag/faiss_index`.

Step 4: Remove the OpenRouter Gate From Retrieval

The retrieval logic should behave as follows:

- If FAISS index is missing, return {} and let the notebook fallback run.
- If FAISS index exists, retrieve relevant chunks locally.
- If `OPENROUTER_API_KEY` is missing, skip only the optional `rag_answer` synthesis.
- If `OPENROUTER_API_KEY` exists, use it only for LLM-generated summary over retrieved context.

Step 5: Align RAG Output With Agent Contract

The notebook remediation agent should receive chunks shaped like:

```
{
  "title": "...",
  "source": "...",
  "score": 0.82,
  "incident_type": "Database",
  "diagnostics": [...],
  "remediation": [...],
  "validation": [...],
  "safety_notes": [...],
  "content": "...",
  "rag_answer": "..."
}
```

Minimum acceptable implementation:

- If structured fields are empty, parse bullet sections from `content`.
- If parsing is not possible, let `remediation_agent` include concise guidance derived from `content` or `rag_answer`.

Preferred implementation:

- Add a small SOP parser in `src/rag`.
- Extract structured sections during ingestion or retrieval.
- Keep raw chunk text for traceability.

Step 6: Ensure Notebook Imports `src`

When the notebook runs from Colab or from `v7-final/notebooks`, ensure the repo root is on `sys.path`:

```
import sys
from pathlib import Path

REPO_ROOT = Path.cwd()
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))
```

If the current working directory is the notebook folder, resolve `REPO_ROOT` to the IncidentIQ repository root.

Step 7: Run Notebook With RAG Enabled

The notebook already supports:

```
options={"enable_rag": True}
```

Once RAG is fixed, the runtime flow should retrieve FAISS context and pass it into the incident agents.

Step 8: Add Smoke Tests

Add a lightweight integration test using one known incident sample ZIP. Assert:

- Clusters are found.
- `rag_context` is non-empty.
- Retrieved sources come from `src/rag/knowledge_base`.
- Remediation actions are not only generic fallback actions.
- Exported JSON includes `rag_context`.

Pending Work Backlog

Priority 0:

- Fix KNOWLEDGE_BASE path.
- Allow FAISS retrieval without OPENROUTER_API_KEY.
- Build the FAISS index.
- Make RAG chunks useful to remediation_agent.

Priority 1:

- Add notebook-to-RAG smoke test.
- Add explicit sys.path handling for Colab and notebook execution.
- Add error handling for missing dependencies and missing FAISS index.
- Include retrieved RAG source filenames in exported incident reports.

Priority 2:

- Extract Pydantic models into src/models.
- Extract ingestion and log parsing into src/ingest.
- Extract signal rules into src/signals.
- Extract clustering into src/clustering.
- Extract agents into src/agents.
- Extract reporting and exports into src/reporting.
- Extract evaluation into src/evals.

Priority 3:

- Build a real LangGraph runner under src/graph.
- Add conditional routing for fallback mode, critic review, retries, and human approval.
- Add FastAPI or application entry point.
- Add real Slack, Jira, GitHub, or n8n adapters behind approval gates.

Recommended Target Architecture

Raw uploads

- > Safe input loader
- > Log parser and normalizer
- > Signal extraction engine
- > Evidence clusterer
- > RAG retriever
- > Classifier agent
- > Timeline agent
- > RCA agent
- > Remediation agent
- > Critic agent
- > Human approval gate
- > Notification and ticket previews
- > Reports and evals

Definition Of Done

The integration is complete when:

- The notebook can run with `enable_rag=True`.
- FAISS retrieval works without an LLM key.
- Optional LLM synthesis works when `OPENROUTER_API_KEY` exists.
- Retrieved SOPs and runbooks improve remediation output.
- Outputs cite both log evidence and RAG source files.
- Hidden ground truth remains eval-only.
- The same behavior is available through a modular `src/` pipeline.